

```

`timescale 1ns / 1ps
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 22.08.2026 14:32:45
// Design Name:
// Module Name: top_tb
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////

module top_tb ();
    // =====
    // DUT signals
    // =====

    reg i_clk;
    reg i_dat_in;
    reg i_rst;
    wire o_pattern_valid;

    // =====
    // Test statistics
    // =====

    integer tests_run;
    integer tests_passed;
    integer errors;

    // =====
    // Reference FSM
    //
    // 0 = RESET
    // 1 = saw 0
    // 2 = saw 01
    // 3 = saw 00
    // 4 = saw 001
    // 5 = saw 010
    // =====

    integer ref_state;

    localparam R_RESET = 0;
    localparam R_0     = 1;
    localparam R_01     = 2;
    localparam R_00     = 3;
    localparam R_001    = 4;
    localparam R_010    = 5;

    // =====
    // DUT
    // =====

    top dut (
        .i_clk          (i_clk),
        .i_dat_in       (i_dat_in),
        .i_rst          (i_rst),
        .o_pattern_valid (o_pattern_valid)
    );

```

```

// =====
// Clock: 10 ns period
// =====

initial begin
    i_clk = 1'b0;
    forever #5 i_clk = ~i_clk;
end

// =====
// Reset DUT and reference model
// =====

task reset_all;
begin
    i_rst      = 1'b1;
    i_dat_in   = 1'b0;
    ref_state = R_RESET;

    repeat (2) @(posedge i_clk);
    #1;

    if (o_pattern_valid !== 1'b0) begin
        $display("ERROR: o_pattern_valid is HIGH during reset");
        errors = errors + 1;
    end

    i_rst = 1'b0;
    #1;
end
endtask

// =====
// Reference model
//
// This is intentionally separate from the DUT implementation.
//
// When a pattern is detected:
//
//     0010 -> VALID, then RESET
//     0100 -> VALID, then RESET
//
// This makes the detector NON-OVERLAPPING.
// =====

task reference_bit;
    input  bit_in;
    output expected_valid;

begin
    expected_valid = 1'b0;

    case (ref_state)
        R_RESET: begin
            if (bit_in)
                ref_state = R_RESET;
            else
                ref_state = R_0;
            end
        R_0: begin
            if (bit_in)
                ref_state = R_01;
            else
                ref_state = R_00;
            end
    end
end

```

```

R_01: begin
    if (bit_in)
        ref_state = R_RESET;
    else
        ref_state = R_010;
    end

R_00: begin
    if (bit_in)
        ref_state = R_001;
    else
        ref_state = R_00;
    end

R_001: begin
    if (bit_in) begin
        // 0011
        ref_state = R_RESET;
    end
    else begin
        // 0010 detected
        expected_valid = 1'b1;

        // NON-OVERLAPPING
        ref_state = R_RESET;
    end
end

R_010: begin
    if (bit_in) begin
        // 0101 -> last two bits are 01
        ref_state = R_01;
    end
    else begin
        // 0100 detected
        expected_valid = 1'b1;

        // NON-OVERLAPPING
        ref_state = R_RESET;
    end
end

default: begin
    ref_state = R_RESET;
end

endcase

end
endtask

// =====
// Send one bit and check DUT against reference model
// =====

task send_bit;
    input bit_in;

    reg expected_valid;

begin

    reference_bit(bit_in, expected_valid);

    i_dat_in = bit_in;

    @(posedge i_clk);
    #1;

```

```

    if (o_pattern_valid !== expected_valid) begin
        $display(
            "  ERROR: input=%b expected=%b actual=%b",
            bit_in,
            expected_valid,
            o_pattern_valid
        );

        errors = errors + 1;
    end

end
endtask

// =====
// Run one complete test
//
// The sequence is always exactly 8, 10, or 12 bits.
// No ambiguous length handling.
// =====

task run_test;

    input [11:0] sequence;
    input integer length;
    input [127:0] name;

    integer i;

begin

    tests_run = tests_run + 1;

    $display("");
    $display("-----");
    $display("TEST %0d: %s", tests_run, name);
    $display("INPUT: %b", sequence);
    $display("-----");

    reset_all;

    for (i = length - 1; i >= 0; i = i - 1) begin
        send_bit(sequence[i]);
    end

    // Pattern valid must only be a one-clock pulse.
    i_dat_in = 1'b0;

    @(posedge i_clk);
    #1;

    if (o_pattern_valid !== 1'b0) begin
        $display("  ERROR: o_pattern_valid did not return LOW");
        errors = errors + 1;
    end
    else begin
        tests_passed = tests_passed + 1;
        $display("  PASS");
    end

end

end
endtask

// =====
// TESTS
// =====

```

initial begin

```

tests_run      = 0;
tests_passed   = 0;
errors         = 0;

i_clk          = 1'b0;
i_dat_in       = 1'b0;
i_rst          = 1'b0;

// =====
// 1. BASIC DIRECT DETECTION
// =====

run_test(12'b0000000001010, 12, "0010 at end");
run_test(12'b000000000100, 12, "0100 at end");

run_test(12'b00100000, 8, "0010 at beginning");
run_test(12'b01000000, 8, "0100 at beginning");

run_test(12'b00010010, 8, "0010 near end");
run_test(12'b00010100, 8, "0100 near end");

// =====
// 2. NO MATCHES
// =====

run_test(12'b11111111, 8, "All ones");
run_test(12'b00000000, 8, "All zeros");
run_test(12'b10101010, 8, "Alternating 10101010");
run_test(12'b01010101, 8, "Alternating 01010101");

run_test(12'b11110000, 8, "11110000");
run_test(12'b11001100, 8, "11001100");
run_test(12'b1111110000, 10, "Long ones then zeros");
run_test(12'b0000001111, 10, "Long zeros then ones");

// =====
// 3. DETECTION AT DIFFERENT POSITIONS
// =====

run_test(12'b00101111, 8, "0010 at position 1");
run_test(12'b10010111, 8, "0010 in middle");
run_test(12'b11001011, 8, "0010 in middle 2");
run_test(12'b11100100, 8, "0100 near end");

run_test(12'b01001111, 8, "0100 at position 1");
run_test(12'b11010011, 8, "0100 in middle");
run_test(12'b11101001, 8, "0100 in middle 2");

// =====
// 4. BACK-TO-BACK DETECTION
//
// These are particularly important for NON-OVERLAPPING.
// =====

run_test(12'b00100100, 8, "0010 then 0100");
run_test(12'b01000010, 8, "0100 then 0010");

run_test(12'b00100010, 8, "0010 then 0010");
run_test(12'b01000100, 8, "0100 then 0100");

run_test(12'b001001001010, 12, "0010 0100 0010");
run_test(12'b010000100100, 12, "0100 0010 0100");

run_test(12'b001000100010, 12, "0010 repeated");
run_test(12'b010001000100, 12, "0100 repeated");

```

```
// =====
// 5. NEAR MISSES
// =====

run_test(12'b00111111, 8, "0011 instead of 0010");
run_test(12'b00011111, 8, "0001 near miss");
run_test(12'b01011111, 8, "0101 instead of 0100");
run_test(12'b01111111, 8, "0111 near miss");

run_test(12'b00110011, 8, "0011 repeated");
run_test(12'b01010111, 8, "0101 near miss");
run_test(12'b01100110, 8, "01100110");
run_test(12'b00010001, 8, "00010001");

// =====
// 6. OVERLAPPING CANDIDATES
//
// These test that a successful match causes RESET.
// =====

run_test(12'b00100100, 8, "Overlap candidate 00100100");
run_test(12'b01001000, 8, "Overlap candidate 01001000");

run_test(12'b0010010010, 10, "Repeated 0010 candidates");
run_test(12'b0100100100, 10, "Repeated 0100 candidates");

run_test(12'b001001001001, 12, "Dense 0010 candidates");
run_test(12'b010010010010, 12, "Dense 0100 candidates");

run_test(12'b001000100100, 12, "Dense mixed candidates");
run_test(12'b010001001001, 12, "Dense mixed candidates 2");

// =====
// 7. LONG ZERO RUNS
// =====

run_test(12'b0000000010, 10, "Zeros followed by 0010");
run_test(12'b0000000100, 10, "Zeros followed by 0100");

run_test(12'b000000000010, 12, "Long zero run then 0010");
run_test(12'b000000000100, 12, "Long zero run then 0100");

run_test(12'b000001000010, 12, "Zero-heavy sequence 1");
run_test(12'b000010000100, 12, "Zero-heavy sequence 2");

// =====
// 8. LONG ONE RUNS
// =====

run_test(12'b111111110010, 12, "Ones followed by 0010");
run_test(12'b111111110100, 12, "Ones followed by 0100");

run_test(12'b111101110010, 12, "Mostly ones with 0010");
run_test(12'b111101110100, 12, "Mostly ones with 0100");

// =====
// 9. MIXED SEQUENCES
// =====

run_test(12'b101100101010, 12, "Mixed sequence 1");
run_test(12'b110010110100, 12, "Mixed sequence 2");
run_test(12'b100100100010, 12, "Mixed sequence 3");
run_test(12'b011001001100, 12, "Mixed sequence 4");

run_test(12'b101001000110, 12, "Mixed sequence 5");
run_test(12'b110100100011, 12, "Mixed sequence 6");
run_test(12'b100110010010, 12, "Mixed sequence 7");
run_test(12'b011010010100, 12, "Mixed sequence 8");
```

```

// =====
// 10. MULTIPLE MATCHES WITH GARBAGE
// =====

run_test(12'b001011110100, 12, "0010 garbage 0100");
run_test(12'b010011110010, 12, "0100 garbage 0010");

run_test(12'b001010110100, 12, "0010 mixed garbage 0100");
run_test(12'b010001100010, 12, "0100 mixed garbage 0010");

run_test(12'b110010110010, 12, "Separated 0010 patterns");
run_test(12'b110100110100, 12, "Separated 0100 patterns");

// =====
// 11. FINAL STRESS CASES
// =====

run_test(12'b001100100100, 12, "Near miss plus patterns");
run_test(12'b010100100010, 12, "Near miss plus patterns 2");

run_test(12'b000100100100, 12, "Long partial prefixes");
run_test(12'b001001100100, 12, "Patterns separated by 11");

run_test(12'b010010100010, 12, "Dense candidate sequence");
run_test(12'b001010010100, 12, "Dense candidate sequence 2");

// =====
// FINAL REPORT
// =====

$display("");
$display("");
$display("=====");
$display("                      FINAL RESULT");
$display("=====");
$display("Tests run      : %0d", tests_run);
$display("Tests passed  : %0d", tests_passed);
$display("Errors       : %0d", errors);
$display("=====");

if (errors == 0) begin
    $display("");
    $display("***** ALL TESTS PASSED *****");
    $display("");
end
else begin
    $display("");
    $display("***** TEST FAILED *****");
    $display("");
end

$finish;

end

endmodule

```